Intermediate (measure the sequential version) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `architecture_comparison_practice.py` (Intermediate section): this is the first real number behind the Basic exercise's on-paper guess — the sequential design, built and measured stage by stage. **If not:** `supervisor_compare` would have nothing fair to compare against, and "sequential is cheaper" would stay a guess.

Solution

Story: a loose 3-item tuple makes `result[1]` mean "tokens" only if you remember the order; named dict keys can't be misread. **If not:** comparing two runs side by side would invite reading seconds as tokens.

```
# architecture_comparison_practice.py
import time
from langchain_openai import ChatOpenAI

# the chat model every stage calls (Doc05's ChatOpenAI)
model = ChatOpenAI(model="gpt-4o-mini")

def research(topic: str) -> dict:
    start = time.perf_counter()
    prompt = f"Research the top 3 causes of {topic}. List them briefly."
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }

def write(research_text: str) -> dict:
    start = time.perf_counter()
    prompt = f"Write a 3-sentence summary of this:\n\n{research_text}"
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
```

```
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }

# Run the two stages in order, and measure each one.
topic = "climate change"
total_start = time.perf_counter()

research_result = research(topic)
write_result = write(research_result["text"])

total_seconds = time.perf_counter() - total_start
total_tokens = research_result["tokens"] + write_result["tokens"]
r_tokens, r_seconds = research_result["tokens"], research_result["seconds"]
w_tokens, w_seconds = write_result["tokens"], write_result["seconds"]

print(f"research: {r_tokens} tokens, {r_seconds:.2f}s")
print(f"write: {w_tokens} tokens, {w_seconds:.2f}s")
print(f"total: {total_tokens} tokens, {total_seconds:.2f}s")
```

Expected output (exact numbers vary by run):

```
research: 210 tokens, 1.84s
write: 96 tokens, 0.91s
total: 306 tokens, 2.75s
```